

PCyMT RM - Comentarios del asistente

Auditoria, opinion visitante, enfoque alternativo y notas de uso de Cursor

Proyecto: Plataforma de Conservacion y Mitigacion del Trafico de Especies con Realidad Mixta

Autor del proyecto: Pedro Chacolla Rybak - Junio 2026

Respuesta 1 — Auditoría práctica del proyecto

Análisis del monorepo (frontend web → backend → móvil): objetivo, entrelazado de datos, fortalezas, deuda técnica y prioridades. Documentación actualizada en docs/project-audit.md y AGENTS.md sección 17.

Qué es el producto

Juego educativo en el Parque de las Culturas y la Madre Tierra: visitantes descubren 12 animales virtuales con AR; staff gestiona contenido desde panel web. Flujo central: VirtualAsset → Location → Interaction → Analytics / Colección móvil; MapConfiguration sincronizada web→móvil.

Frontend web — bien / mal / cambiar

- Bien: Angular 21, panel completo, mapa canvas avanzado, UX Fases 1-3, tokens semánticos.
- Mal: sin guard de rol; i18n a medias en mapa/animador; código muerto (TableControl, tilemap-editor); Chart.js global.
- Cambiar: AdminGuard; lazy Chart.js; eliminar dead code; verificar endpoint animation-sequence.

Backend — bien / mal / cambiar

- Bien: API REST, Vercel+DB, avatares whitelist, rate limits, config pública.
- Mal: RBAC casi ausente; uploads Multer no persisten en Vercel; tests integración stale (/api/v1/).
- Cambiar: adminOnly middleware; upload directo Supabase; alinear tests; config mutable en BD.

Mobile — bien / mal / cambiar

- Bien: 3 modos AR, ParkMapView, networking robusto, avatares perfil, monorepo stickers.
- Mal: admin duplica web; fragments >1400 LOC; README desactualizado; mapa duplicado vs Angular.
- Cambiar: decidir admin móvil; un camino AR; JSON shared boundary; actualizar README.

Prioridades roadmap

- 1. Seguridad: AdminGuard + adminOnly backend.
- 2. Uploads prod Supabase end-to-end.
- 3. Decisión admin móvil vs solo web.
- 4. Limpieza código muerto.
- 5. i18n + tests integración.

Respuesta 2 — Opinión visitante (caja negra) y fases de deuda

El juego a grandes rasgos

Pokédex del parque: caminar, encontrar 12 figuras, verlas en AR, completar colección. Loop: login → progreso X/12 → Mapa / Juku Go / Colección → AR → interacción registrada → trofeos.

Prueba como visitante — qué funciona

- Meta clara: «Has encontrado X de 12».
- Juku Go es la experiencia más «juego» (exploración + encuentros).
- Celebración al completar 12 — sensación de logro real.
- Fallback RA simple en dispositivos sin ARCore.

Prueba como visitante — qué falla o confunde

- App se llama «Realidad Mixta» pero el botón principal es «Juku Go».
- Sin onboarding: no sé si usar Mapa, Juku Go o Colección.
- Fuera del parque la app parece rota (sin mensaje de geofence).
- Realidad Mixta poco visible para visitantes (card oculta si no eres admin).
- Sin anclas colocadas, RM parece bug, no contenido pendiente.
- Banners «Modo Admin» visibles con cuenta staff — confunde en demo.
- Panel web accesible con cuenta visitante — no debería existir ese camino.

Veredicto producto

Base lúdica sólida (meta + exploración + AR). Falta un solo camino claro, tutorial, y alinear promesa del nombre con lo que vive la mayoría (RA simple / Juku Go).

Deuda dividida en fases

Fase 1 — Puedo jugar sin confundirme: onboarding, naming, geofence, ocultar admin, RM visible, empty states AR.

Fase 2 — Juego enseña: misiones por sección, feedback unificado al encontrar, narrativa educativa.

Fase 3 — Solo staff en consola: AdminGuard, RBAC backend, decidir admin móvil.

Fase 4 — Producción evento: uploads Supabase, anclas en parque, CI tests reales.

Fase 5 — Pulido: i18n mapa, identidad visual, dead code, tests mapa/AR.

Fase 6 — Escalar (post-TFG): JSON shared mapa, partir fragments, flavors visitor/staff.

Respuesta 3 — Cómo haría la app yo (enfoque alternativo)

Respeto por tu enfoque actual

Monorepo + Node + Angular admin + Kotlin ARCore + mapa custom + Vercel/Supabase es defendible para TFG solo con cliente real. No reescribiría todo; podría producto y endurecer decisiones.

Dos productos claros

PCyMT Explorer (visitante, Android): un loop explorar → AR → coleccionar 12. PCyMT Console (staff, web): CRUD + mapa + analytics. Sin «modo admin» en APK visitante.

Stack que elegiría (2026)

- Backend: TypeScript + NestJS/Fastify + Prisma + PostgreSQL + Supabase Storage (uploads día 1).
- Admin: Angular 21 o React; Material; Chart.js lazy; sin SSR/PWA en panel interno.
- Móvil: Kotlin + Jetpack Compose; Navigation Compose; 4 tabs; Hilt completo; Coil/Retrofit.
- Shared: park-boundary.geojson + park-sections.json — una verdad, dos renderers (web canvas + ParkMapView).
- AR: 1 modo principal (explorador GPS) + RM opcional en POIs icónicos; cloud-anchor-cli para staff.
- DevOps: pnpm/Nx monorepo; GitHub Actions; Sentry; OpenAPI.

Qué NO cambiaría de lo tuyo

- Kotlin nativo para AR.
- Mapa custom del parque.
- PostgreSQL + REST + JWT.
- Monorepo con shared/uploads.
- Panel Angular para staff.
- Analytics e interacciones.

Prioridad sin reescribir

- Renombrar: «Explorador PCyMT»; RM como feature.
- Juku Go = corazón del juego.
- Quitar admin del APK visitante.
- RBAC + AdminGuard + uploads Supabase.
- JSON shared del parque + onboarding 3 pantallas.

Veredicto

El desvío principal fue de producto (demasiados modos AR y roles en un binario), no técnico. La demo al jurado: «Explorador del parque — 12 especies, staff en consola web».

Respuesta 4 — Este documento y scroll en Cursor

Sobre este PDF

Archivo generado: docs/PCyMT-Comentarios-Asistente.pdf (script: scripts/generate-assistant-comments-pdf.py).
Contiene resumen fiel de las cuatro respuestas del chat sobre auditoría, visitante, enfoque alternativo e instrucciones de uso de Cursor.

Scroll suave en el chat de Cursor

En Cursor, las flechas $\uparrow$ $\downarrow$ a menudo navegan entre bloques/mensajes del hilo cuando el foco está en el panel de conversación, no desplazan el scroll como en un documento. Por eso «saltan comentarios» en lugar de bajar línea a línea.

Formas de desplazarte suavemente

- Rueda del ratón o gesto de dos dedos en trackpad sobre el área del chat — scroll continuo (lo más natural).
- Barra de scroll lateral del panel de chat — arrastrar con el ratón.
- Page Down / Page Up — salto por pantalla dentro del panel.
- Shift + rueda del ratón — en algunos entornos Linux desplaza horizontal/vertical según configuración.
- Clic primero dentro del texto de una respuesta larga, luego rueda — asegura que el scroll va al contenedor correcto.
- Evita tener el foco en la caja de input abajo si quieres scroll; las flechas ahí mueven el cursor del texto que escribes.

Atajos útiles (basados en VS Code / Cursor)

- Ctrl + Home / Ctrl + End — inicio/fin del documento o panel con foco (según contexto).
- Ctrl + $\uparrow$ / Ctrl + $\downarrow$ — a veces mueven línea en editor; en chat pueden no aplicar.
- Esc — quita foco del input y permite que el siguiente scroll vaya al historial.

Si las flechas siguen saltando mensajes

- Es comportamiento de diseño del chat (navegación por turnos), no un bug de tu sistema.
- Para lectura larga: exportar o usar este PDF; o copiar la respuesta a un archivo .md en el proyecto.
- Ajustes: Cursor Settings → buscar «scroll» / «chat» por opciones de smooth scroll del editor (el chat puede no exponer todas).
- Alternativa: panel Agent en ventana más ancha o en monitor secundario para más área de scroll con rueda.

Regenerar el PDF

Desde la raíz del repo: `python3 scripts/generate-assistant-comments-pdf.py`